Claim Complexity Algorithm

Project Requirements and Specifications

Penser North America



WSU Student Team

Kevin Herrera, Brian Leung, Ishaan Joshi

Sep 30th, 2026

Table of Contents

I. Introduction 2

I.2. Background and Related Work 2

I.3 Project Overview.....2

I.4 Client and Stakeholder Identification and Preferences 3

II. Team Members – Bios and Project Roles 4
II.1. Kevin Herrera – Team Lead 4
II.2. Brian Leung 5
II.3. Ishaan Joshi 5

III. System Requirements Specification 6

III.1. Use Cases 6
III.2. Functional Requirements 9
III.2.1. Claim Analysis and Complexity Model 9
III.2.2. User Interface and Integration 11

III.3. Non-Functional Requirements 13

III.4. User Stories 15
III.4.1. User Stories 15
III.4.2. Traceability Matrix 18

III.5. Standards 18

IV. System Evolution 19

V. Architecture Design 20

V.1. Overview.....20

V.2. Subsystem Decomposition.....21

V.2.1 Date Ingestion and Preprocessing.....22

V.2.2. Complexity Label Generation.....23

V.2.3. Model Training and Benchmarking.....25

V.2.4. Inference and Scoring Engine.....26

V.2.5. Backend API and Authentication 27

V.2.6. Web Dashboard 28

VI Data Design 28

VII User interface Design 30

VIII Constraints 30

IX Glossary 31

X References 32

I. Introduction

This project will develop an artificial intelligence system for analyzing insurance claims. The system will produce two primary outputs: a claim complexity score, representing the expected difficulty of processing a claim, and an expected payout, estimating the potential financial cost of the claim.

The system will consist of machine learning models, a web-based interface, and an API. Customers and insurance employees will use the web interface, while insurance companies will be able to integrate the system into larger applications through the API. The system is intended to provide data-driven decision support rather than replace the judgment of insurance professionals.

I.2. Background and Related Work Workers compensation pays for medical treatment and lost wages when an employee is hurt on the job. Third party companies like Penser manage these claims for employers and insurers, and an examiner handles each claim from the time it is filed until it closes. Each claim has fields the examiner enters.

I.3 Project Overview The Problem There is currently no standardized data driven way to measure how complex a workers compensation claim is. Complexity is estimated subjectively and it depends on many factors like the injury type, industry time loss, return to work status, and vocational rehabilitation needs legal activity fraud or malingering risks either by an injured worker or a medical provider and the settlement trajectory. No single person can easily weigh all these consistencies across 10,000 claims. Issues Assignment and staffing: Without a reliable measure, TPAs cannot deliberately match claims to experienced examiners or anticipate staffing needs as their client portfolio grows. Cost Visibility: Employers clients have limited insight into which claims will become costly which makes it harder to set reserves and design safety programs. Misleading proxies: Examiner activity, such as the number of touch points, is easy to measure, but it can reflect an individual examiner's habits rather than how difficult the claim really is. Treating it as complexity would build that bias into any model. Defensibility: Time loss decisions need to be accurate and defensible, and an informal approach makes it harder to justify the claims.

The Goal: The project will deliver an AI-assisted framework that combines claim characteristics into a composite complexity score and a companion cost score for each claim. It is being built first for Washington State claims and designed so it can later expand to other states.

Objectives:

Define and score complexity: Build a score that reflects how long a claim is likely to take and how much work it requires without treating examiner activity alone as complexity.

Estimate cost: Use closed claims with known final costs as anchors then work backward to find which characteristics are associated with the costs.

Allow tuning: Support adjustable weights including an inflation related weight so cost estimates account for changing claim costs.

Communicate Confidence: Shows examiners a 0 – 100 confidence percentage and gives leadership and engineering a graphical view.

Stay interpretable: Make the factors behind each score explainable.

Data: We will begin with about 10,000 claims from the last five years, roughly 7 MB, covering industries such as agriculture, construction, police, fire, and maritime work. The data includes examiner-entered fields such as claim start date and actual cost, plus narrative diary notes that are scrubbed of identifying information and tokenized before we receive them. Penser will send the column headers first, and the full five-year notes dump is expected around early October because Penser's claims system is moving to a new platform. If the data is not diverse enough or accuracy is too low, Penser can provide older claims going back to the 1990s. Every participant signs a non-disclosure agreement because the data includes legal and medical information.

Desired outcomes and Requirements: The final system should output a complexity score, a cost score, and a confidence value for each claim. Claims are expected to arrive gradually, about one per minute on average. The system should handle three per minute to cover examiners working at the same time, with an inference ceiling of 10 minutes. The model will be retrained about every six months. Per the proposal, the deliverables are AI/ML integration, a dashboard with visualizations, a backend system, and a database solution. Development starts locally, with AWS or SageMaker evaluated later for hosting. The working budget for AWS, Claude, and related needs is about $300 to $500, and our mentor, Joell Parks, is to be notified if costs get close to $500. A retraining budget will be proposed once retraining is ready.

Intended Users and Expected Impact: TPAs: assign claims to appropriately experienced staff and plan capacity as their client portfolio grows.

Employers: see which claim types and job classifications drive high complexity outcomes informing safety program and reserve strategy.

Insurance brokers: get better insight into underwriting needs by client industry. Injured workers: get closer oversight on the claims that need it most.

State of Washington: Also identified as an intended user in the proposal. The broader outcome is a more accurate and defensible way to calculate claim complexity and time loss turning that awareness into measurable operational and safety results.

Timeline: Work is planned across three sprints: planning and research, initial model training, and retraining or evaluation, which includes checking whether the 10,000 claims are enough. Sprint 1 covers reviewing the column headers and deciding which model approaches fit the data. Sprints are due October 5, November 5, and December 5. The team meets with Penser online every week, and Joell has offered regular stand-ups during the sprints.

I4. Client and Stakeholder Identification and Preferences

Primary Client and Mentors

Our primary client is Penser North America, represented by our mentor and primary contact, Joell Parks. Keno Peck and Ritvik Sharma are additional contacts at Penser. Keno works directly with our team on the data, including the column headers, the claims dataset, and the sanitized

notes, and on the development plan. Ritvik is coordinating our Claude access. Joell guides the project, set our working budget of about $300 to $500, and wants to be told if costs get close to $500.

Development Team: Kevin Herrera, Brian Leung, and Ishaan Joshi are responsible for analyzing claim complexity, building the complexity, cost, and confidence outputs, and checking whether the initial dataset is enough.

Stakeholders and Expected Users

The primary users will be TPAs and claims examiners. They need a quick, consistent read on how complex a claim will be and what it will likely cost, so claims can go to examiners with the right experience and workload can be planned as the client portfolio grows. Examiners will see a 0 to 100 confidence percentage next to each score, which matches Penser's other platforms.

Penser leadership and engineering: They will get a graphical view of the results and will maintain the system, including retraining the model about every six months. Before a retraining budget is set, they want us to propose an approach and cost estimate.

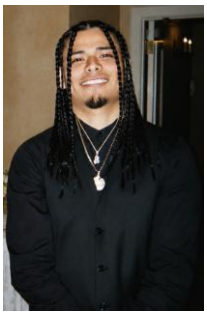
Employer-clients: They want to see which claim types and job classifications lead to high-complexity claims, which helps with safety programs and reserve strategy.

Insurance brokers: They want better insight into underwriting needs based on their clients' industries.

Injured workers: They are affected indirectly. The system aims to bring closer oversight to the claims that need it most. Because the data comes from their claims, identifying details are scrubbed and the notes are tokenized before we receive them, and everyone on the project signs a non-disclosure agreement.

II Team Members Bios and Project Roles

II.1 Kevin Herrera – Team Lead



Kevin Herrera is a Computer Science student at Washington State University with experience developing full-stack and mobile applications. His technical background includes React Native, Expo, Next.js, TypeScript, Hugging Face, PostgreSQL, and REST APIs, with a focus on authentication, database design, and role-based access control. Through independent software development, Kevin has built applications for invoicing, tournament management, and fieldservice operations, including software deployed for live business testing. He also has experience developing AI-driven applications, including a WSU hackathon project that received the Best Use of Presage award. For this project, Kevin contributes experience in application

development, backend systems, and integrating machine learning functionality into practical software systems.



II.2 Brian Leung

Brian Leung is a Computer Science student at Washington State University with a minor in Mathematics and experience in machine learning, data science, and distributed systems. His technical background includes Python, C/C++, SQL, PyTorch, Hugging Face, NumPy, pandas, Neo4j, and Apache Spark. Brian has developed machine learning projects involving natural language processing and neural networks, including a tweet sentiment classifier using parameter-efficient fine-tuning and a handwritten digit classifier built from scratch. He has also worked with large-scale data analytics, developing a distributed Amazon co-purchasing analysis system using Neo4j and Apache Spark. As a Teaching Assistant for programming and data structures courses, he has experience helping students apply programming, data structures, and algorithm analysis concepts. For this project, Brian contributes experience in machine learning, data analysis, algorithm development, and software engineering.



II.3 Ishaan Joshi

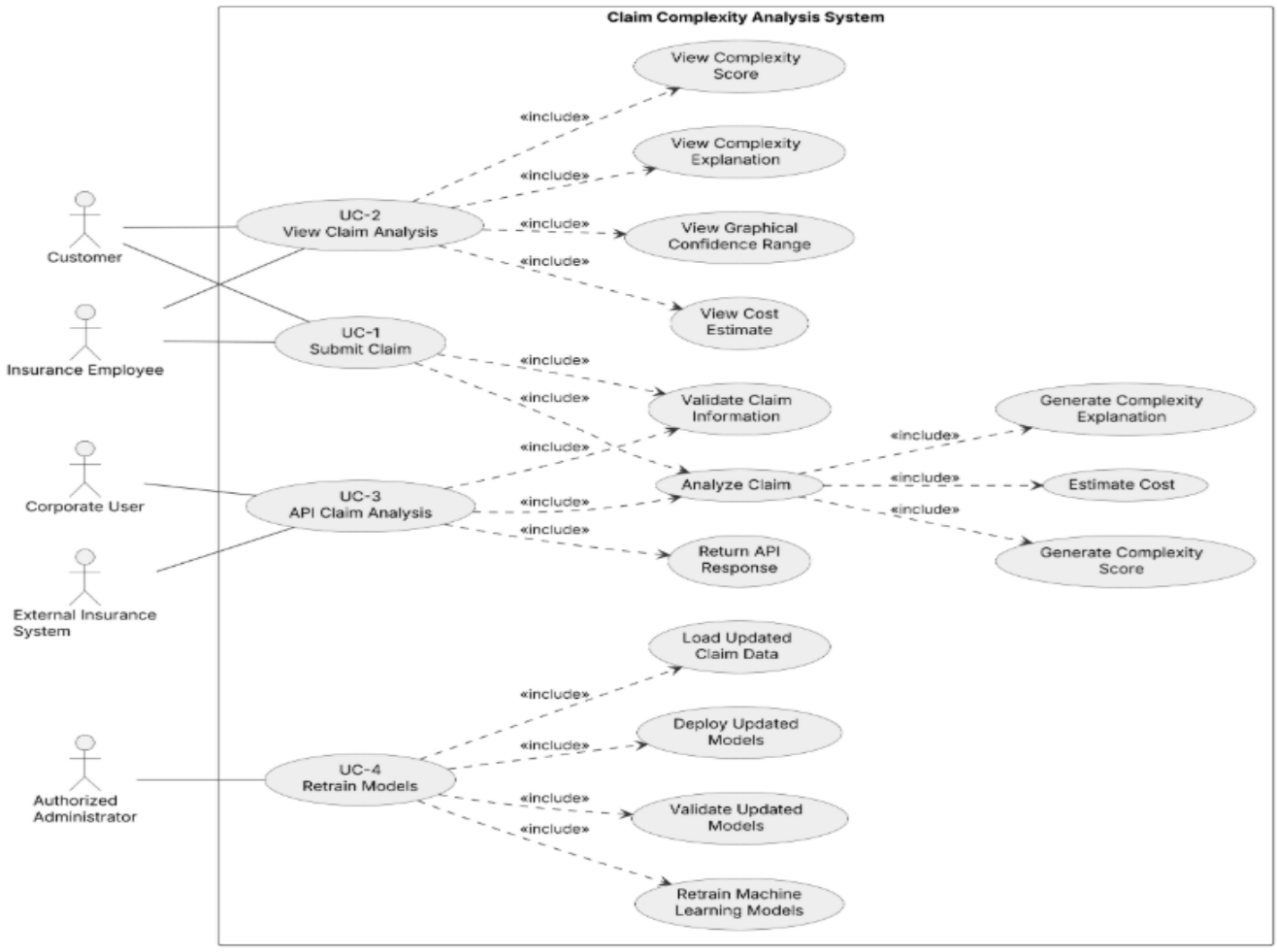
Ishaan Joshi is a Software Engineering student at Washington State University pursuing a minor in Mathematics, with experience in software development, formal methods, and artificial intelligence. His technical background includes C/C++, Python, finite automata, GitHub/GitLab, and experience with AI and machine learning. Ishaan has worked on projects involving accessible navigation software, application development, and symbolic graph analysis, including a BDD-based symbolic model checker for evaluating graph reachability and quantified first-order logic statements. His coursework includes algorithms, software engineering, data structures, automata and formal languages, probability and statistics, linear algebra, and graph theory. For this project, Ishaan contributes experience in software development, algorithms, mathematical modeling, and formal methods.

III. System Requirements Specification

The system requirements describe the functionality and properties necessary to provide claim analysis to customers, insurance employees, and corporate users.

III.1. Use Cases

This section describes how users and external systems interact with the claim complexity analysis system. Down below is Figure 1 and it shows the four use cases in a single diagram and the table that follows describes each use case its actor and its related requirements.


Claim Complexity Analysis System
View Complexity Score
View Complexity Explanation
View Graphical Confidence Range
View Cost Estimate
Validate Claim Information
Generate Complexity Explanation
Analyze Claim
Estimate Cost
Generate Complexity Score
Return API Response
Load Updated Claim Data
Deploy Updated Models
Validate Updated Models
Retrain Machine Learning Models
UC-2 View Claim Analysis
UC-1 Submit Claim
UC-3 API Claim Analysis
UC-4 Retrain Models
«include»
Customer
Insurance Employee
Corporate User
External Insurance System
Authorized Administrator

ID	Name	Actor	Description	Related Requirements
UC-1	**Submit Claim**	Customer or Insurance Employee	An authenticated user submits claim information through the web interface. The system validates the information and submits it for analysis. The analysis produces a complexity score, expected payout, and explanation of the complexity score	FR-1, FR-2, FR-3, FR-4, FR-5, NFR-1
UC-2	**View Claim Analysis**	Customer or Insurance Employee	The user views the complexity score, expected payout, graphical confidence range, and an explanation of the factors contributing to the complexity score	FR-2, FR-3, FR-4, FR-5, NFR-2
UC-3	**API Claim Analysis**	Corporate User / External System	An external insurance system submits claim information through the API and receives the model's predictions and applicable explanation information.	FR-6, NFR-3
UC-4	**Retrain Models**	Authorized Administrator	The system's machine learning models are retrained using updated claim data. The current plan is to retrain approximately every six month	FR-7, NFR-4

III.2. Functional Requirements

This section lists the functions that the system must perform. Section II.2.1 helps cover the claim analysis and the complexity model which includes the complexity scores, sub-scores, and the explanation cost estimate as well as the confidence range. III.2.2 helps cover the user interface API model and the retraining and inflation adjustment

III.2.1. Claim Analysis and Complexity Model

ID	Requirement	Description	Source	Priority
FR-1	Claim Data Processing	The system must accept and process insurance claim information for training and inference. The initial dataset is expected to contain approximately 10,000 records covering five years and approximately 7 MB of data.	Project stakeholders need to analyze historical and new insurance claims.	Level 0
FR-2	Complexity Score	The system must produce a claim complexity score representing the expected difficulty of processing a claim. The complexity score must be calculated as a weighted average of multiple sub-scores.	Project stakeholders; need to estimate claim difficulty using multiple contributing factors.	Level 0
FR-3	Complexity Sub-Scores	The system must calculate the individual sub-scores used to produce the overall complexity score. The sub-scores must be combined according to configurable weights. The sub-scores are expected to cover factors such as time loss, medical conditions, return to work, vocational rehabilitation, litigation, permanent partial	Project stakeholders; need to represent different sources of claim complexity.	Level 0

		disability, settlement, and total claim cost. Examiner touch points must not be used on their own to decide that a claim is complex because a high number of touch points can come from how an examiner works rather than from the claim itself. The final sub-scores and their starting weights will be decided after the team reviews the claims data, using closed claims with known final costs as a reference.		
FR-4	Complexity Explanation	The system must provide an explanation of the complexity score that identifies the factors contributing to the score. The explanation should communicate which sub-scores and other claim characteristics have the greatest effect on the overall result. Factors that come from the sanitized diary notes may be mapped back to readable categories so they can be understood in the explanation. The exact layout of the explanation will be decided during the design phase.	Penser examiners, leadership, and engineering need to understand why the system produced a particular complexity score.	Level 0
FR-5	Expected Cost Estimate	The system must produce an expected cost estimate for an analyzed claim based on historical claim information and relevant claim characteristics. The cost estimate must be produced alongside the complexity score for every claim. Closed claims with known final costs must be used	Project stakeholders; need to estimate potential claim cost	Level 0

		as the reference for these estimates.		
FR-6	Confidence Range	The system must display prediction confidence as a graphical range for the cost estimate and other applicable predictions. For examiners, confidence must also be shown as a percentage from 0 to 100 to stay consistent with other platforms Penser uses. Leadership and engineering must be able to view the graphical range. The method used to calculate confidence will be decided during the design phase.	Project stakeholders; need to communicate prediction uncertainty.	Level 0

III.2.2. User Interface and Integration

ID	Requirement	Description	Source	Level
FR-7	Web Interface and API	The system must provide a web interface for authenticated Penser examiners, leadership, and engineering, and an API that allows claim data to be sent to the system and results to be returned. The web interface must allow users to submit claims and view analysis results, including the complexity score, cost estimate, confidence, and complexity explanation. Access to	Penser examiners, leadership, and engineering need direct access to results, and claims	**Level 0**

		both the web interface and the API must require authentication through AWS Cognito, which is part of the reference stack Penser specified. The specific API endpoints and data formats will be decided during the design phase.		
FR-8	Model Retraining	The system must support periodic model retraining using new available claim data. The current target is about every six months. This schedule may be reevaluated later to account for inflation and changes in new arriving claims. If accuracy is too low or the data lacks coverage, the training data may be expanded to include older claims going back as far as the 1990s. When retraining is ready, the team will give Penser a proposed approach and estimated cost so the budget can be reviewed. The model approach will be chosen after the team reviews the available data during the planning and research sprint.	Project stakeholders; need to maintain model relevance as new data becomes available	Level 1
FR-9	Inflation Adjustment	The system must account for changes in monetary values when using historical claim costs. The current approach is to use weighted inflation adjustments. The inflation weight must be tunable along with the other weights. This adjustment also needs to account for historical cost differences if older claims going back to the 1990s are added to the data.	Project stakeholders; need for meaningful cost estimates across different time periods.	Level 1

.

III.3. Non-Functional Requirements

This section describes the qualities and constrains the system must mee, including the performance usability, security, scalability and the cost and maintainability

ID	Category	Description
NFR-1	Performance	The system shall support approximately one claim per minute under normal usage and up to three claims per minute under higher usage. The maximum expected inference time is 10 minutes.
NFR-2	Usability and Explainability	The system shall present the complexity score, expected payout, graphical confidence range, and complexity explanation clearly to customers and insurance employees.
NFR-3	Security	The system shall require authentication for Penser examiners, leadership, and engineering. The system should only use diary notes that Penser has sanitized, with identifying information removed and dictionary words tokenized. The authentication technology, user permissions, and API security will be decided during the design phase.
NFR-4	Scalability	The system shall be capable of future integration into larger insurance systems through its API.
NFR-5	Cost	The system shall be developed within an expected total budget of approximately $300–$500, including approximately $100–$300 in AWS credits.
NFR-6	Maintainability	The system should separate the machine learning models, web interface, and API sufficiently to allow individual components to be updated. The complexity score weights should be configurable without requiring major changes to the system. Development should begin locally, and AWS or SageMaker will be evaluated later for hosting so the system can move to the cloud. The exact

		deployment, infrastructure, and testing requirements will be decided during the design phase.

III.4. User Stories and Traceability Matrix

US-1 — Customer Submits a Claim User

Story:

As a customer, I want to submit my claim information, so that the system can analyze my claim. **Success Conditions:**
Given I am an authenticated customer with valid claim information,

When I submit my claim,

Then the system accepts the claim and generates a complexity score, expected payout, confidence range, and explanation of the complexity score. **Exceptional Conditions:**
Given my claim information is incomplete or invalid,

When I submit the claim,

Then the system informs me that the claim cannot be processed and identifies the required information.

Related Requirements: FR-1, FR-2, FR-3, FR-4, FR-5, FR-6, NFR-1, NFR-3

US-2 — Customer Views Claim Analysis User

Story:

As a customer, I want to view my claim analysis, so that I can understand the expected complexity and cost of my claim. **Success Conditions:**
Given my claim has been successfully analyzed,

When I view the claim results,

Then the system displays the complexity score, contributing factors, expected payout, graphical confidence range, and explanation of the complexity score. **Exceptional Conditions:**
Given an analysis is unavailable or cannot be completed,

When I attempt to view the results,

Then the system informs me that the analysis is unavailable.

Related Requirements: FR-2, FR-3, FR-4, FR-5, FR-6, NFR-2

US-3 — Claims Handler Analyzes a Claim

User Story:

As a claims handler, I want to analyze a claim, so that I can better understand its complexity and expected cost. **Success Conditions:**

Given I am an authenticated claims handler with valid claim information,

When I submit the claim for analysis,

Then the system provides the complexity score, contributing sub-scores, expected payout, confidence range, and explanation of the complexity score.

Exceptional Conditions:
Given the claim information is incomplete or invalid,

When I submit the claim,

Then the system identifies the invalid or missing information and does not generate an analysis.

Related Requirements: FR-1, FR-2, FR-3, FR-4, FR-5, FR-6, NFR-1, NFR-3

US-4 — Corporate User Integrates the System User

Story:

As an insurance company, I want to access the claim analysis system through an API, so that I can integrate claim analysis into existing software. **Success Conditions:**
Given the external system provides valid claim information through the API,

When the API receives the request,

Then the system processes the claim and returns the applicable analysis results, including the complexity score, expected payout, and explanation information.
Exceptional Conditions:
Given the API request contains invalid or incomplete information,

When the request is received,

Then the system returns an error indicating that the request cannot be processed.

Related Requirements: FR-7, NFR-1, NFR-4

US-5 — Administrator Retrains the Model

User Story:

As an administrator, I want to retrain the machine learning models, so that the system can incorporate newly available claim data. **Success Conditions:**
Given updated claim data is available,

When the administrator initiates model retraining,

Then the system trains the models using the updated data and makes the updated models available for future analysis. **Exceptional Conditions:**
Given the training data is invalid or unavailable,

When the administrator attempts to retrain the models,

Then the system reports the problem and does not replace the existing models.

Related Requirements: FR-8, NFR-6

III.4.2. Traceability Matrix

Stakeholder Need	User Story	Functional Requirement	Non-Functional Requirement	Priority
Analyze claims	US-1, US-3	FR-1	NFR-1	Level 0
Measure claim complexity	US-2, US-3	FR-2, FR-3	NFR-2	Level 0
Explain claim complexity	US-2, US-3	FR-4	NFR-2	Level 0
Estimate claim cost	US-2, US-3	FR-5	NFR-2	Level 0
Understand prediction uncertainty	US-2, US-3	FR-6	NFR-2	Level 0
Integrate with insurance systems	US-4	FR-7	NFR-4	Level 0
Maintain model relevance	US-5	FR-8	NFR-6	Level 1
Account for changing costs	US-2, US-3	FR-9	NFR-6	Level 1

III.5. Standards

The system will follow established software engineering, accessibility, artificial intelligence, and professional ethics standards to support reliable, maintainable, and responsible development. IEEE 830-1998, Recommended Practice for Software Requirements Specifications, is used as a guideline for structuring and documenting the system requirements. The standard supports the organization of the project's functional requirements, non-functional requirements, use cases, and traceability between stakeholder needs and system requirements [1].

The system's web interface will follow W3C Web Content Accessibility Guidelines (WCAG) 2.2, with the project targeting Level AA where applicable [2]. Accessibility considerations will apply to the presentation of claim complexity scores, expected payout estimates, confidence ranges, and complexity explanations. The interface will use readable text, sufficient color contrast, clear labels, and keyboard-accessible controls. Accessibility will be evaluated during interface testing before deployment.

Because the system uses machine learning to analyze insurance claims, the project will follow the principles of the NIST Artificial Intelligence Risk Management Framework (AI RMF 1.0) [3]. The framework is relevant to the project's requirements for model reliability, transparency, explainability, and risk management. In particular, the system will provide an explanation of the factors contributing to a claim's complexity score rather than presenting the score as an unexplained prediction. Model performance will also be evaluated using held-out data during development and retraining. Specific fairness metrics and thresholds will be determined during model development.

The project will also follow the ACM Code of Ethics and Professional Conduct when designing and implementing the system [4]. This is particularly relevant because insurance claims can contain sensitive information and the system's predictions could influence how claims are handled. The system is therefore intended as a decision-support tool rather than a replacement for human judgment. Access to claim information will require authentication, and users will only be provided access appropriate to their role. The team will also evaluate the model for potential sources of bias as part of model validation.

For data security, the project will consider the NIST Cybersecurity Framework (CSF) 2.0 when designing authentication, access control, data protection, and API security [5]. Customers will only be permitted to access their own claims, while authorized insurance employees will be able to access claims appropriate to their role. Data transmitted between the web interface, API, and backend services will use HTTPS/TLS. Specific encryption-at-rest and data-retention configurations will be determined during implementation.

IV. System Evolution

The initial system will use a defined machine learning architecture to generate claim analysis. The complexity component is expected to use multiple sub-scores representing different sources of claim complexity. These sub-scores will be combined using a weighted-average approach to produce the overall complexity score.

The exact sub-scores, weights, model implementation, and scoring scales remain under development. As additional claim data becomes available, the sub-score definitions and weights may be adjusted based on model evaluation and stakeholder feedback. The system is expected to be retrained approximately every six months.

Future versions may incorporate additional complexity factors, improved prediction models, additional outputs, and integration with larger insurance systems. The system should preserve the ability to update individual components without requiring a complete redesign.

The machine learning approach has not been chosen yet. The team will select it during the planning and research sprint after reviewing the column headers and data provided by Penser.

V. Architecture Design

V.1. Overview

The claim complexity analysis System uses a layered architecture with four layers the layers include presentation layer, an application layer an analysis layer and lastly a data layer. Each layer depends on the layer below it. Requests from examiners leadership engineering and external insurance systems enter through the top layers. While the machine learning work and stores data are kept in the lower layers.

We choose layered architecture for three reasons. First, it separates the machine learning models from the web interface and the API which NFR-6 requires so that the individual components can be updated without redesigning the system. Second, the models will be retrained about every 6 months of FR8, and a layered design lets a new model version be swapped in without changing the interface or the API. Third, it helps support the planned move from local development to AWS, because each layer can be deployed desperately. Inside the analysis layer the training work follows a pipe-and-filter pattern, where data moves through ingestion labeling training and evaluation in a fixed order

Layers and Components

Presentation layer – Web Dashboard: A react web application where examiners submit claims and view the complexity score, cost estimate, confidence and explanation and where leadership and engineering view graphical results.

Application Layer – Backend API and Authentication: A python REST API that receives requests from the dashboard and from external insurance systems, checks authentication through AWS Cognito validate claim data calls the scoring engine and stores the results.

Analysis Layer: Contains the inference and scoring engine which produces results for a single claim. The data ingestion and preprocessing subsystem, which cleans and prepares claim data. The complexity label generation subsystem which creates training labels for complexity and the model training and benchmarking pipeline which trains and compares the publishes models.

Data Layer: A PostgreSQL database that stores claims sanitized diary notes generated labels predictions and configurable weights and a model registry that stores versioned model weights.

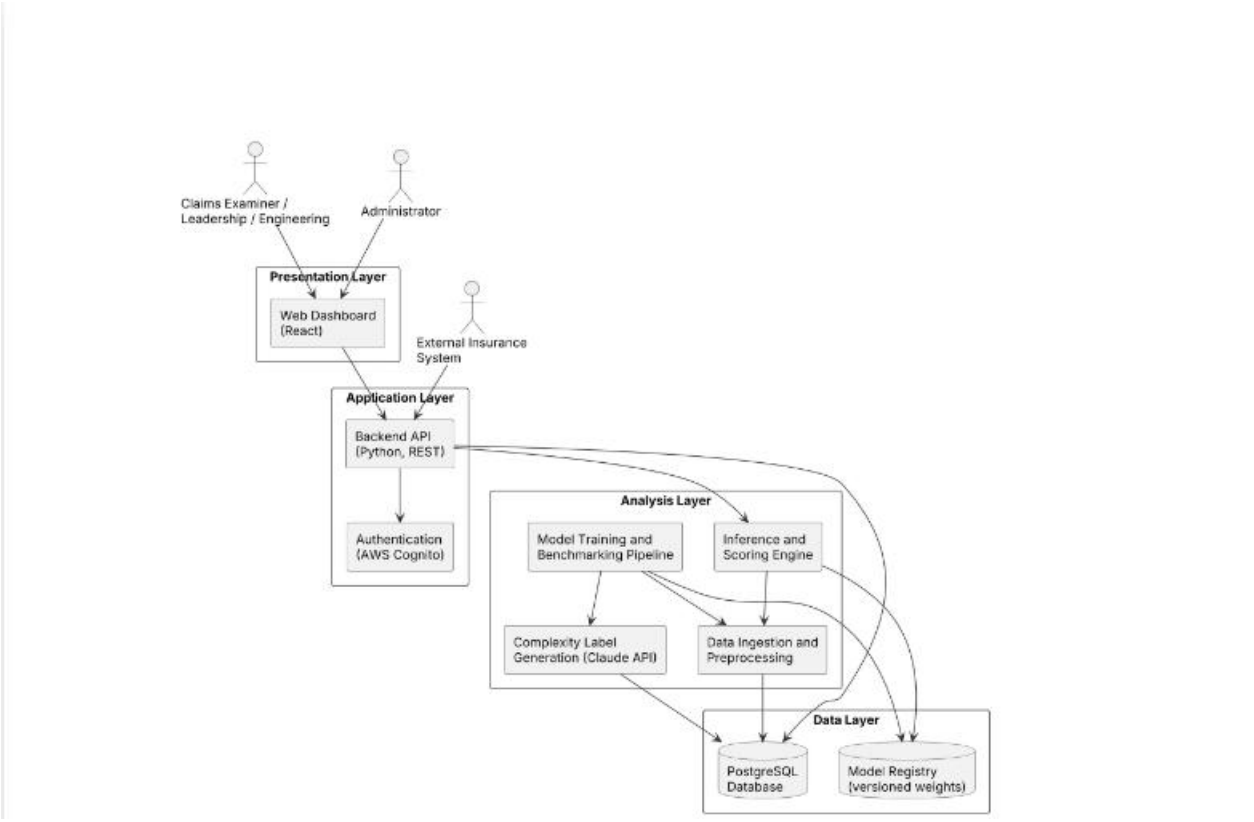


Figure 2. Layered architecture of the claim complexity analysis system.

V.2.: Subsystem Decomposition

The system will be divided into 6 subsystems so that each one has a single clear responsibility and can be developed by one team member. The subsystems were grouped by that they do. Either serving a users (Web dashboard Backend API), producing predictions (inference and scoring engine) and building models Data ingestion and preprocessing complexity label generation model training and benchmarking.

The decomposition keeps cohesion high because each subsystem functions all relate to the same task. For example, everything involving model training lives in one subsystem. It keeps coupling low because the subsystem communicates only through the small set of interfaces shown below in Figure 3. The dashboard never talks to the models directly, and the scoring engine only receives new models through the Publish Model Version interface. This means the training pipeline can change models, and the dashboard can change layouts without affecting other subsystems.

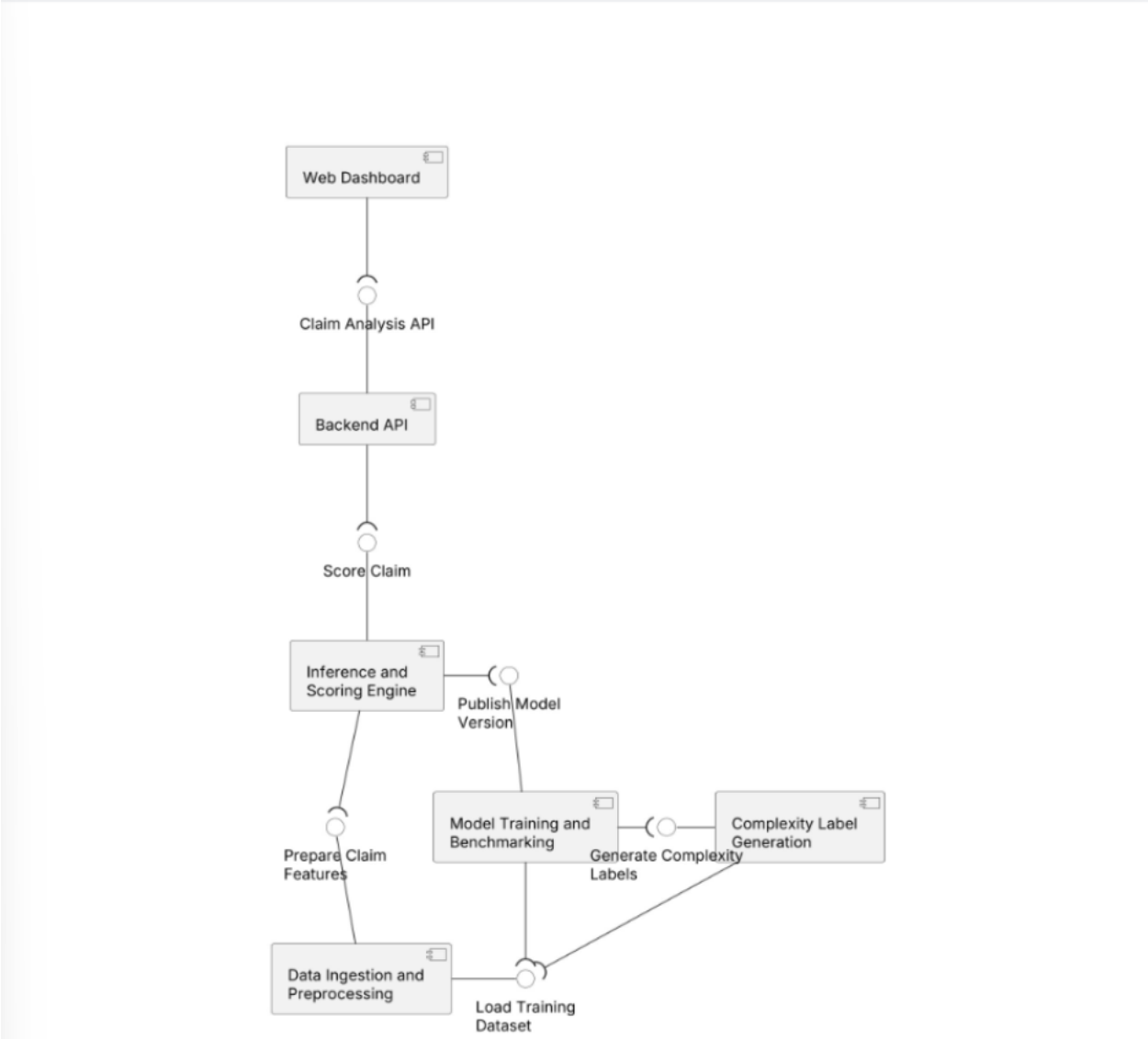


Figure 3: Component diagram showing provided and required interfaces.

V.2.1 Data ingestion and Preprocessing

a) Description

This subsystem loads the penser's claim data and turns it into a form the model can use it for reading the structured claim fields and help sanitized the diary notes, for cleaning and validating the records for adjusting the historical costs for inflation the (FR-9) splitting the data into training validation and test sets and preparing the features for a single claim at the inference time.

b) Concepts and Algorithms Generated

The initial dataset is about 10,000 claims for the last 5 years. Each claim has a structured field, such as a claim start date and the actual cost and the free text diary notes. We considered three ways to help represent the claim.

Structured fields only: Simple and fast but ignores the diary notes which hold most of the detail about litigation treatment and the return to work.

Diary notes only: Captures the narrative but loses exact values such as dates and costs.

Combined (selected): The diary notes are passed to the language model and the structured fields are added as extra features. This uses all the available information.

Industry BERT-Insurance like other BERT-based models can read 512 tokens at a time and claims diary notes are often longer. We will split long notes into overlapping chuck's to encode each chunk and combine the chuck results by averaging. Historical costs that will be adjusted to current dollars using an inflation index with an adjustable inflation weight stored with the other weights. Data will be split by claim not by note, so the same claim never appears in both the training and the test sets. The closed claims with known final costs will be used as a reference for evaluating the cost estimates.

C) Interface description

Services Provided:

1. **Service Name**: Prepare Claim Features

   Service provided to: Inference and Scoring Engine

   Description: Takes one claim's structured fields and diary notes as an input. Validates the record applies inflation adjustment chunks the notes and returns a feature object ready for the models. Returns a list of missing or invalid fields if the claim cannot be processed.

2. **Service Name**: Load Training Dataset

   Services provided To: Model training and benchmarking the complexity of label generation.

   Description: Takes a dataset version as an input and returns the cleaned inflation-adjusted training validation and test splits.

   Services Required:

   Read access to the claims and diary notes tables in the PostgreSQL database.

V.2.2. Complexity Label Generation

A) **Description**

Penser's data includes final payouts, but it does not include a human-assigned complexity score for each claim. This subsystem creates complexity labels so that a complexity model can be trained. It sends each training claim to the Claude API, which returns a score for each complexity sub-score and a short reason. The levels are stored in the database for training.

B) **Concepts and Algorithms Generated**

We considered three ways to get complexity labels:

Manual labeling by examiners: most trustworthy but not practical for 10,000 claims within the project timeline.

Proxy labels from the data: For example using claim duration or total cost as a complexity. This is cheap but does not match the Penser's defnition of complexity and the touch points and cannot be used on their own(FR-3).

Large language model labels (selected): The Claude API reads each claim and how the scores and each sub-score using a fixed rubric . This can label the whole dataset at a low cost and follows the sub-score definitions.

Becuase there is no human-labeled data, the final complexity model will learn to reproduce what Claude produces. This a form of knowledge distillation where a smaller model learns from a large one. To help reduce a risk of copying Claude's mistakes we will use a fixed prompt and rubric for each claim and compare the labels against any sort of proxy signals such as a claim duration and final cost. And ask Penser examiners to help review a small random sample. To help only sanitized diary notes that are approved by the API and the labeling cost will be tracked agisnt the project budget.

C) **Interface Description**

Services Provided:

1. **Service name**: Generate Complexity Labels

   Service provided to: Model Training and Benchmarking

   Description: Takes a set of training claims as an input. Returns a score for each sub-score such as a time loss, medical involvement and return to work or a vocational rehabilitation lilgation permanent partial disability and settlement and a short reason and the rubric used.

Services Required:

Load Training Dataset(Data ingestion and Preprocessing).

Claude API(external), accessed through Penser-approved credentials.

V.2.3 Model Training and Benchmarking

a) **Description**

This subsystem trains the models that estimate payout and complexity and compares the candidate models and publishes the selected model version to the model registry. It also runs the periodic retraining planned for about every 6 months (FR-8).

b) **Concepts and Algorithms Generated**

We are considering four open-weight models that are specialized for insurance text. The table below summarizes them.

Model	**Parameters**	**Approx Storage**
Industry-BERT-Insurance	110M	~420MB
InsureLLM-4B	4B (nominal base)	~8-10 GB(FP16)
ClaimSense AI	7B	~13-14GB(FP16)
Turtlemunt Insurance Embeddings	109M (BGE-base architecure)	~438MB

Table 1 Candidate models

Our current top choice is Industry-BERT-Insurance. It is small enough to fine-tune and retrain within our budget and it can run inference well within the 10-minute limit in NFR-1. The 4B and 7B models may be more accurate but they need far more memory and GPU time, which makes retraining every 6 months more expensive.

Each model will be evaluated through the following steps:

1. Benchmark with training only a regression layer, download the open weight model and estimate the payout and complexity directly to get a baseline.
2. Benchmark after LoRA fine tune the model with LoRA which trains a small number of added parameters instead of the whole model then estimate payout and complexity again.
3. Train the payout estimate: use supervised learning since the final payouts are provided in the data closed for claims.

4. Train the complexity estimate: train on the sub-score labels from the complexity label generation subsystem, then combine the sub-scores into one score using the configurable weights (FR-2,FR-3).
5. Select and train the final model.

For the final model we will choose between two approaches, A big model trimmed down to a small model (distillation) is likely to give better accuracy. A small model trained directly is less expensive to retain. The choice will depend on the benchmark results and the retraning cost estimate we give Penser. Models will be compared on held-out test data, using mean absolute error for the payout and agreement with the labels for complexity.

c) Interface Description

Services Provided:

1. **Service name**: Publish Model Version

   Service provided to: Inference and Scoring Engine

   Description: Saves the trained payout and complexity models their evaluation results and a version number to the model registry. The scoring engine uses the newest approved version. A model is only published if it passes evaluation so a failed retraining does not replace the current model.

Services Required:

Load Training Dataset(Data ingestion and Preprocessing).

Generate Complexity Labels (Complexity Label Generation)

V.2.4 Inference and Scoring Engine

a) Description

This subsystem produces the analysis for a single claim it loads the current model version and runs the payout and complexity models and helps combine the sub-scores into the overall complexity score and calculates the confidence and builds the explanation.

b) Concepts and Algorithms Generated

The overall complexity score is a weighted average of the sub-scores. The weights are read from the database so they can be changed without retraining (NFR-6). For confidence we considered reporting a single model score using a quantile regression to predict a low and a high value and running an ensemble of models and measuring their spread. We plan to use quantile regression for the cost estimate because it directly produces the graphical range that leadership and engineering need, and we will convert the width of that range into 0 –100 percentage examiners see (FR-6). The final method will be confirmed during the design. For the explanation (FR-4), the engine reports which sub-scores contributed most to the weighted score, and maps factors found in the diary notes back to the readable categories.

c) Interface Description

Services Provided:

Service name: Score Claim

Service provided to: Backend API

Description: Takes a validated claim as input. Returns the complexity score, each sub-score and weight, the cost estimate with its low and high range, the 0 – 100 confidence value, the explanation, and the model version.

Services Required:

Prepare Claim Features (Data ingestion and Preprocessing).

Publish Model Version (Model Training and Benchmarking), through the model registry.

V.2.5 Backend API and Authentication

a) Description

This subsystem is the single entry point to the system. It receives claim submissions and result requests from the Web Dashboard and from external insurance systems, checks that the user is authenticated, validates the requests and calls the inference and the scoring engine and saves the results to the database.

b) Concepts and Algorithms Generated

We chose a REST API written in Python so it can share code and libraries with the machine learning subsystem. Authentication uses AWS Cognito which is part of the reference stack Penser specified for each request is checked for a valid token and the user's role. Because inference can take up 10 mintues (NFR-1) long requests will be handled asynchronously: the API return a reqwuesr ID right away and the client checks back for the result.

C)Interface Description

Service Name: Claim Analysis API

Service provided to: Web Dashboard external insurance systems

Description: Endpoints to submit a claim check the status of an analysis, retrieve results and for administrators start retraining and update weights. The inputs and outputs are JSON. Invalid or incomplete requests return an error that list the problem fields.

Services Required:

Score claim (Inference and Scoring Engine).

AWS Cognito (external) for authentication.

Read and write access to the PostgreSQL database.

V.2.6 Web Dashboard

a) Description

This subsystem is the user interface for the Penser and the examiners in leadership and engineering and the administrators. It lets users sign in submit claim's view results and for administrators to manage weights and retraining.

b) Concepts and Algorithms Generated

The dashboard will be built in react, which is part of Penser's preffered stack. Different users need different views: examiners need a fast view with a score of 0 – 100 confidence value, while leadership and enginerring needs charts. The dashboard therefore shows a different default view based on the user's role. The layout will follow WCAG 2.2 Level AA guidelines.

c) Interface Description

Services Provided:

None to other subsystems. The dashboard provides user interfaces to people.

Services Required:

Claim analysis APU (Backend API and Authentication).

VI. Data Design

The system stores its data in a PostgreSQL database and stores trained models in a separate model registry. The table Below lists the main database tables.

Table	Purpose	KEY
Claims	One row per claim with the structured fields from Penser.	Claim_id, start_date, close_date, status, industry, actual_cost, inflation_adjusted_cost
diary_notes	Sanitized, tokenized diary notes linked to a claim.	note_id, claim_id, note_date, note_text
complexity_labels	Sub-score labels generated by the Claude API for training.	Claimid, subscorename, score, reason, rubric_version
predictions	Results produced for each analyzed claim.	Prediction_id, claim_id complexity_score, cost_estimate, cost_high, confidence, explanation, model_version, created_at
sub_scores	Each sub_score for a prediction.	predictionid, subscorename, value, weight
Weights_config	Current adjustable weights for sub-scores and inflation	Weight*name, value, updated*by, updated_at
Model_versions	Metadata for each published model	model_version, modelname, traningdate, evaluationresults, is_active
analysis_requests	Status of each submitted analysis request	request*id, claim*id, requested*by, status,* submitted*at,* completed_at

Table 2: Main database tables

User accounts are managed by AWS Cognito, so the database only stores each user's Cognito ID where it is needed, such as in requested by and updated_by. Model weights are stored as versioned files in the model registry with the matching row in model_versions. Inside the system a claim moves through a few temporary data structures, a validate claim record a feature object with the encoded note chucks and structured fields, and a result object that is saved to the predictions and sub_scores tables. All data stays within approved storage, and only sanitized diary notes are stored or sent to an external service.

VII. User Interface Design

Users interact with the system through the Web Dashboard which they reach in a web browser and after signing in with their Penser account through AWS Cognito. External insurance systems use the AOU instead of the dashboard. The main screens are listed below along with the use cases from the Requirements and Specification document that they support.

Sign-in screen: A form for entering or uploading claim information and diary notes. The form highlights missing or invalid fields before submission. Supports the UC-1(Submit Claim).

Claim Analysis screen: Shows the complexity score, the cost estimate and the 0-100 confidence value at the top. Below that a bar chart shows each sub-score and its weight a range chart shows the low expected and high cost, and the explanation lists the factors that contributed most to the score. Supports UC-2(View Claim Analysis).

Leadership Overview screen: Charts that summarize results across claims such as how many claums fall into each complexity level and how cost estimates are distributed. Supports UC-2.

Administration screen: Lets authorized administrators adjust sub-score and inflation weights see which model version is active and start retraining. Supports UC-4(Retrain Models).

API users follow the same flow as UC-3(API Claim Analysis): they send a claim in JSON to receive a request ID and retrieve the results when the analysis is complete. Installation and configuration for the developmental team will be documented in the project README, including setting up the Python environment the database connection and the Cognito settings.

VIII Constraints

The design must meet the following constraints:

Cost: The total budget is about $300 – 500, including about $100 - $300bin AWS credits, and covers cloud hosting, GPU time, and Claude API usage. Our mentor must be notified if costs approach $500. This is the main reason for preferring a small model.

Data privacy and legal: The claims include medical and legal information. Everyone on the project signs a non-disclosure agreement, only diary notes sanitized by Penser may be used, and data sent to external services must be approved by Penser.

Data availability: There are no human-labeled complexity scores so complexity labels must be generated and checked. The dataset of about 10,000 claims may not be enough. Older claims back to the 1990s may be added if needed.

Data format: Diary notes are tokenized before we receive them, which may limit how much a pretrained language model can understand them.

Performance: The system must handle one claim per minute normally and up to three per minute at peak, with a maximum inference time of 10 minutes.

Hardware and Storage: The larger candidate models need about 8 – 14 GB of storage and a GPU with enough memory to run, which affects both hosting cost and retraning time.

Interoperability: Each score must be explainable so the design must favor approaches whose results can be broken down into sub-scores and factors.

Schedule: Work is dived into three sprints that end of October 11th sprint 1 November 8th sprint 2 December 8th.

V. Glossary

Claim Complexity Score: A numerical estimate representing the expected difficulty of processing an insurance claim. The score is expected to be calculated as a weighted average of multiple complexity sub-scores.

Complexity Sub-Score: A score representing an individual factor contributing to claim complexity, such as litigation chance, medical involvement, return to work, rehabilitation, or other complications.

Expected Payout: The system's estimate of the potential financial cost associated with an insurance claim.

Confidence Range: A graphical representation of the uncertainty associated with a model prediction.

Complexity Explanation: Information describing the factors and sub-scores that contributed to the overall complexity score.

Inference: Using a trained machine learning model to produce a prediction for new claim data.

Retraining: Updating a machine learning model using additional or newer training data.

API: An interface that allows external software systems to communicate with the claim analysis system.

VI. References

[1] Institute of Electrical and Electronics Engineers, *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*, IEEE, 1998.

[2] World Wide Web Consortium (W3C), *Web Content Accessibility Guidelines (WCAG) 2.2*, W3C Recommendation, 2024.

[3] E. Tabassi, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, National Institute of Standards and Technology, 2023.

[4] Association for Computing Machinery, *ACM Code of Ethics and Professional Conduct*, ACM, 2018.

[5] National Institute of Standards and Technology, *The NIST Cybersecurity Framework (CSF) 2.0*, NIST Cybersecurity White Paper 29, 2024.

[6] B.Bruegge and A.H Dutoit, Object-oriented Software enginerring Using UML, Patterns, and Java, 3rd ed. Upper Saddle River, NJ USA: Prentice Hall,2010.

[7] J.Devlin, M.-W Chang,K.Lee,and K.Toutanova "BERT: Pre- traning of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL- HLT, 2019, PP. 4171-4186.

[8] E.J. Hu et al., "LoRA" Low- Rank adaptation of Large Language Models," in Proc. Int. Conf. Learning Representations (ICLR), 2022.

[9] G. Hinton, O. Vinyals, and J.Dean, "Distilling the knowledge in a Neural Network," arXiv:1503.02531,2015

[10] R. Koenker and G. Bassett "Regression Quantiles," Econmetrica, vol 46, no.1, pp.33-59,1978.